Retro Snake Game Using FPGA

Gunja Jaswanth
Dept. of Electrical Engineering
Shiv Nadar Institution of Eminence
Delhi NCR, India
2310110476

Ravuri Leela Venkata Sai Krishna
Dept. of Electrical Engineering
Shiv Nadar Institution of Eminence
Delhi NCR, India
2310110576

Dasari Yashaswi Praneeth Reddy
Dept. of Electrical Engineering
Shiv Nadar Institution of Eminence
Delhi NCR, India
2310110456

Pranav Krishna Palanisamy
Dept. of Electrical Engineering
Shiv Nadar Institution of Eminence
Delhi NCR, India
2310110556

***Abstract*—This project focuses on the design and implementation of a classic Snake game using an FPGA board. The objective is to develop an interactive gaming system by leveraging digital hardware design and FPGA technology's real-time processing capabilities. The game allows users to control the snake's movement using input controls, with the output displayed on an external screen. The system is structured using a modular design, with separate functional blocks handling game logic, display generation, user input processing, and random target generation. These modules work together to manage core functionality, including snake movement, collision detection, score tracking, and dynamic screen updates. A key aspect of the project is the generation of display signals, and the synchronization required to render the game in real time. The design ensures smooth operation by coordinating timing signals with game logic updates. The implementation highlights important FPGA concepts, including parallel processing, hardware-level control, and efficient module integration. Overall, the project successfully demonstrates the application of FPGA-based system design to develop a real-time, interactive game, providing practical exposure to digital design principles, hardware description languages, and system-level integration.**

***Index Terms*—FPGA, Verilog, VGA Controller, Digital Logic Design, Real-Time Processing, Parallel Computing**

I. Introduction

Field Programmable Gate Arrays (FPGAs) serve as a critical bridge between software programming and physical hardware fabrication, allowing for highly deterministic, parallel execution of tasks. This project capitalizes on these hardware capabilities to reconstruct the classic "Snake" arcade game natively in logic gates.

Unlike modern software engines that render graphics via sequential instruction execution on a CPU or GPU, this FPGA-based implementation requires generating raw synchronization signals and color vectors directly for a Video Graphics Array (VGA) monitor. The continuous real-time rendering demands strict adherence to timing protocols while simultaneously updating game mechanics asynchronously in the background.

Under supervision of Prof. Sonal Singhal and Dr. Rohit Singh, Department of Electrical Engineering.

By segmenting the architecture into discrete Verilog modules—clock dividers, input handlers, display generators, and logic controllers—the system takes full advantage of concurrent hardware execution. This methodology ensures that pixel drawing, random number generation, and user inputs are processed instantaneously without the latency typically introduced by operating system interrupts.

II. System Architecture

The system architecture adopts a multi-layered hardware approach, segregating the high-speed rendering logic from the slower-paced game mechanics.

Elaborated RTL design schematic illustrating the synthesized gate-level interconnections.

A. Clock Management Layer

Standard FPGA development boards typically operate on a master clock of 50 MHz or 100 MHz. However, standard VGA resolution (640×480 at 60Hz) necessitates a specific pixel clock, and human-playable mechanics require an even slower frequency. Thus, the clock is bifurcated:

- **VGA Clock Divider:** A simple 2-bit counter scales the system clock down to match the required 25 MHz pixel clock for VGA output.

- **Update Clock Generator:** The `UpdateClock` module generates a `move_tick` pulse. Using a 23-bit counter parameterized with a `COUNT_MAX` of 2,000,000, it artificially creates a low-frequency strobe that acts as the heartbeat for snake movement, preventing the snake from moving at millions of pixels per second.

B. VGA Signal Generation

The `VGAgenerator` module is the visual core of the system. It tracks the electron beam's current position using horizontal (`xCount`) and vertical (`yCount`) registers. To achieve a 640×480 resolution, the counters actually sweep across a larger 800×525 grid to account for the front porch, sync pulse, and back porch blanking intervals required by analog CRT monitors and modern LCDs scaling the protocol.

C. Hardware Input and Debouncing

User commands are captured through the `ButtonInput` module. Mechanical switches inherently suffer from contact bounce, meaning a single press might generate multiple high-/low transitions. The module uses a two-step synchronization process. First, it registers the raw input on the fast system clock. Second, it only commits the direction change (LEFT, RIGHT, UP, DOWN) when the slow `move_tick` goes high. This effectively masks out bouncing and prevents the user from registering illegal reverse inputs within a single game frame.

III. Hardware Component Analysis

The hardware integration revolves around the Digilent Nexys 4 FPGA board and a standard VGA monitor.

A. Nexys 4 FPGA Board Constraints

The project relies on specific hardware interfaces provided by the Nexys 4 Rev. B board. The mapping is rigorously defined in the Xilinx Design Constraints (XDC) file.

TABLE I
HARDWARE INTERFACE TO NEXYS 4 PIN MAPPING

Component	FPGA Pin	Signal Purpose
`clk`	E3	Master 100 MHz oscillator
`red[3:0]`	A4, C5, B4, A3	4-bit Red color vector
`green[3:0]`	A6, B6, A5, C6	4-bit Green color vector
`blue[3:0]`	D8, D7, C7, B7	4-bit Blue color vector
`h_sync`	B11	Horizontal Synchronization
`v_sync`	B12	Vertical Synchronization
`reset (BTNC)`	E16	Game Reset Logic
`u (BTNU)`	F15	Snake UP control
`d (BTND)`	V10	Snake DOWN control
`l (BTNL)`	T16	Snake LEFT control
`r (BTNR)`	R10	Snake RIGHT control

IV. Methodology and Logic Implementation

The software implementation translates game states into Register-Transfer Level (RTL) logic.

A. State Representation and Memory

The snake's body coordinates are stored in two shift registers: `snakeX` (10-bit arrays) and `snakeY` (9-bit arrays). The dimensions are chosen mathematically; $2^{10} = 1024$ is sufficient to store horizontal coordinates up to 640, while $2^9 = 512$ is sufficient for vertical coordinates up to 480. The length dynamically increases, controlled by the integer `snake_length`, up to a parameterized `MAX_LENGTH` of 10 blocks.

B. Coordinate Shift Logic

On every `move_tick`, if the `gameOver` flag is false, the system executes a parallel shift. A Verilog `for`-loop propagates the coordinates of each body segment to the segment behind it:

$$\text{snakeX}[i] \leftarrow \text{snakeX}[i-1]$$

$$\text{snakeY}[i] \leftarrow \text{snakeY}[i-1]$$

The head (`snakeX[0]`, `snakeY[0]`) is then updated based on the current directional vector (UP, DOWN, LEFT, RIGHT). The step size is fixed to `CELL_SIZE` (10 pixels), ensuring movement operates on a locked grid.

C. Target Generation via LFSR

Random apple placement is necessary for unpredictability. Generating true randomness in digital hardware is impossible without analog noise sources. Therefore, the `Random` module utilizes a Linear Feedback Shift Register (LFSR). Running constantly on the fast `VGA_clk`, it shifts bits and XORs specific taps:

$$\text{randX} \leftarrow \{\text{randX}[8:0], \text{randX}[9] \oplus \text{randX}[6]\}$$

When the snake consumes an apple, the instantaneous, unpredictable value of the LFSR is modulo-divided by the grid constraints (60×44) and scaled by `CELL_SIZE` to plot the next target.

V. Core RTL Implementations

To illustrate the efficiency of hardware-level game development, critical portions of the Verilog implementation are documented below.

A. VGA Generation Code

The precise tracking of display areas is handled continuously, generating blanking pulses when the counters exceed the visible boundaries.

```
always @(posedge VGA_clk) begin
    if (xCount == H_TOTAL-1) begin
        xCount <= 0;
        if (yCount == V_TOTAL-1) yCount <= 0;
        else yCount <= yCount + 1;
    end else xCount <= xCount + 1;

    displayArea <= (xCount < 640) && (yCount < 480);
    hSync <= (xCount >= 656 && xCount < 752);
    vSync <= (yCount >= 490 && yCount < 492);
end
```

Listing 1. VGA Timing Generator

B. Collision Detection and Pixel Driving

Collision is treated as pure combinatorial logic. It continuously checks if the calculated `nextSnakeX` or `nextSnakeY` falls outside the predefined `BORDER_W`. The VGA output block then multiplexes colors based on whether the current `xCount`/`yCount` pixel corresponds to an apple, snake body, or border.

```
always @(posedge VGA_clk) begin
    if (!displayArea) begin
        red <= 0; green <= 0; blue <= 0;
    end else if (gameOver) begin
        red <= 4'hF; green <= 0; blue <= 0;
    end else begin
        red <= {4{apple}};
        green <= {4{snakeBody}};
        blue <= {4{border}};
    end
end
```

Listing 2. RGB Pixel Driving Logic

VI. System Testing and Validation

Prior to bitstream generation and hardware programming, the design underwent rigorous RTL simulation to verify functional intent.

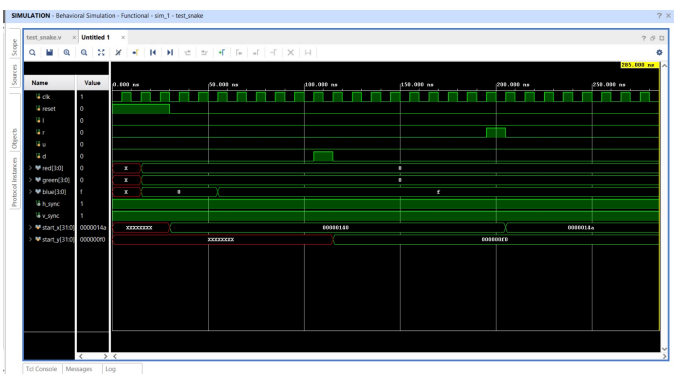

Fig. 1. Behavioral simulation waveform demonstrating clock cycles, input triggers, and VGA synchronization signals.

A. Testbench Strategy

The `test_snake` module was constructed to simulate the asynchronous logic interactions. To avoid waiting millions of clock cycles in simulation, the `UpdateClock` parameter was drastically downscaled (`COUNT_MAX = 4`).

B. Functional Validation

The testbench verified that:

- Asserting the `d` (down) button successfully increased the `snakeY` register upon the next `move_tick`.
- Asserting the `r` (right) button successfully increased the `snakeX` register.
- Default reset states correctly initialized the snake size and default coordinates.

VII. Results

The synthesized design was evaluated for hardware efficiency, area utilization, and power consumption upon deployment to the target architecture.

A. Resource Utilization

Post-synthesis implementation mapping confirmed that the logic was highly efficient for the target Artix-7 architecture.

Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF	BRAM	URAM	DSP
constrs_1	synth_design Complete!												3471	1472	0	0	0
constrs_1	write_bitstream Complete!	1.038	0.000	0.128	0.000		0.000	0.122	0	84 CW, 5 Warn			3450	1481	0	0	0

Fig. 2. Post-implementation resource utilization summary showing LUT and FF counts.

As shown in Fig. 2, the design utilized 3,450 Lookup Tables (LUTs) and 1,481 Flip-Flops (FFs). This confirms the array-shifting logic and LFSR targets mapped cleanly to distributed logic without over-consuming Block RAM (BRAM). The implementation successfully routed with zero timing or setup violations, confirming a stable design.

B. Power Analysis

An assessment of the thermal constraints and energy requirements was conducted based on the final routed netlist.

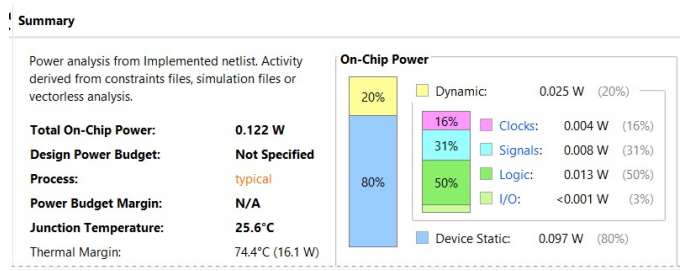

Fig. 3. On-chip power analysis summary for the synthesized design.

Power analysis, detailed in Fig. 3, indicated an exceptionally low total on-chip power consumption of 0.122 W. Dynamic power accounted for 0.025 W, largely driven by the high-frequency pixel clock logic handling VGA rendering, while device static power remained stable at 0.097 W. This low power footprint is ideal for continuous operation in a game application setting.

VIII. Individual Contribution

The success of this comprehensive design was achieved through parallel development and strict modular assignments:

- **Gunja Jaswanth:** Designed the core game logic, including coordinate movement matrices, scoring mechanisms, array shifting, and border collision detection.
- **Ravuri Leela Venkata Sai Krishna:** Developed the VGA display system generator, configured the blanking intervals, and handled the continuous on-screen pixel rendering logic for the snake body and targets.

- **Pranav Krishna Palanisamy:** Managed global timing control, designed the specific clock dividers, and integrated all individual modules into the top-level entity for proper synthesis.
- **Dasari Yashaswi Praneeth Reddy:** Implemented input debouncing logic, developed the LFSR randomizer, and performed rigorous RTL testing, timing simulation, and documentation compilation.

IX. Conclusion

This project successfully achieved its goal of recreating a real-time, interactive Retro Snake Game purely through FPGA hardware synthesis. The modular design approach effectively separated parallel processing tasks—such as VGA pixel rendering, clock generation, input debouncing, and coordinate matrix updates. By managing display signals and game logic concurrently, the system provides an entirely deterministic gaming experience free of software-induced latency. Furthermore, the implemented design is highly efficient, utilizing minimal on-chip power (0.122 W). Ultimately, this implementation highlights the vast potential and superiority of Field Programmable Gate Arrays for real-time control, providing foundational exposure to digital logic design using Hardware Description Languages.

Acknowledgment

We thank Dr. Rohit Singh and Prof. Sonal Singhal of the Department of Electrical Engineering at Shiv Nadar Institution of Eminence for their technical guidance and supervision throughout this project.

X. GitHub Link

The complete source code, hardware constraints, and Verilog implementation for this project are publicly available on GitHub:

FPGA Snake Game Repository

References

[1] Digilent, "Nexys 4 FPGA Board Reference Manual," [Online]. Available: https://digilent.com/reference/_media/reference/programmable-logic/nexys-4/nexys4_rm.pdf. [Accessed: Apr. 22, 2026].

[2] Nandland, "VGA Introduction: Driving Test Patterns to a VGA Monitor," [Online]. Available: https://nandland.com/project-9-vga-introduction-driving-test-patterns-to-vga-monitor/. [Accessed: Apr. 22, 2026].

[3] Wikipedia contributors, "Linear-feedback shift register," Wikipedia, The Free Encyclopedia. [Online]. Available: https://en.wikipedia.org/wiki/Linear-feedback_shift_register. [Accessed: Apr. 22, 2026].